What is C?

C is a general-purpose programming language created by Dennis Ritchie at the Bell Laboratories in 1972.

It is a very popular language, despite being old. The main reason for its popularity is because it is a fundamental language in the field of computer science.

C is strongly associated with UNIX, as it was developed to write the UNIX operating system.

Why Learn C?

- **It is one of the most popular programming languages in the world**
- If you know C, you will have no problem learning other popular programming languages such as Java, Python, C++, C#, etc, as the syntax is similar
- **C is very fast, compared to other programming languages, like Java and Python**
- **C is very versatile; it can be used in both applications and technologies**

Difference between C and C++

- **C++ was developed as an extension of C, and both languages have almost the same syntax**
- **The main difference between C and C++ is that C++ support classes and objects, while C does not**

C Quickstart

Let's create our first C file.

Open Codeblocks and go to File > New > Empty File.

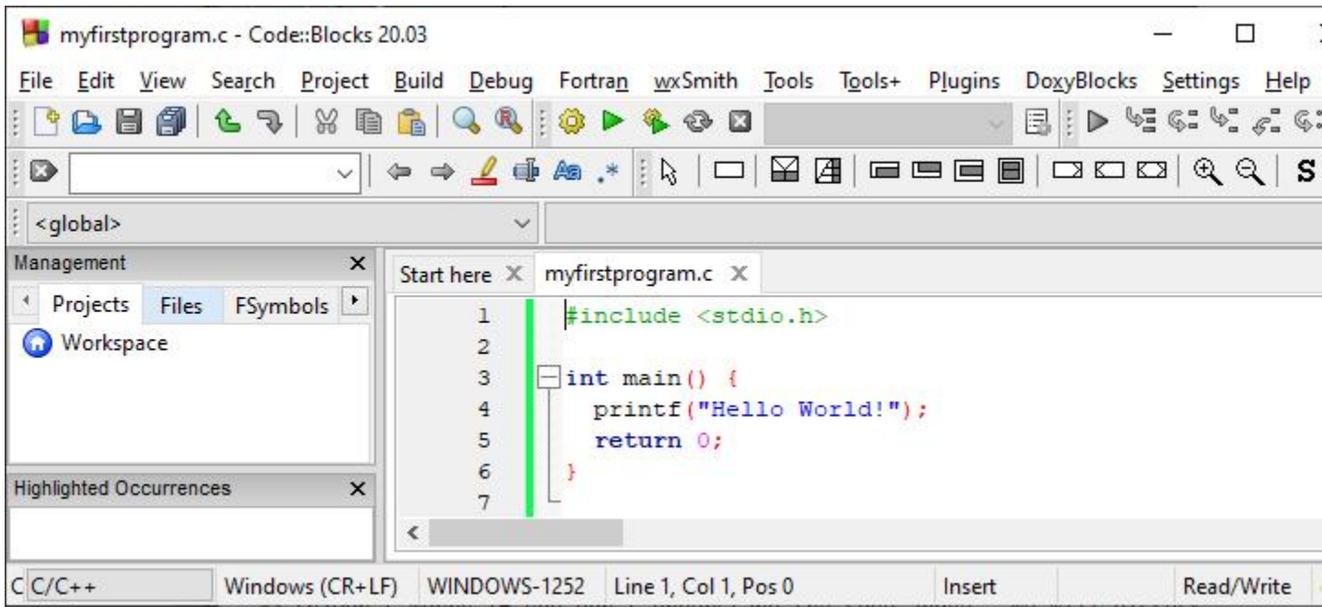
Write the following C code and save the file as myfirstprogram.c (File > Save File as):

myfirstprogram.c

```
#include <stdio.h>

int main() {
  printf("Hello World!");
  return 0;
}
```

In Codeblocks, it should look like this:



Then, go to Build > Build and Run to run (execute) the program. The result will look something to this:

```
Hello World!
Process returned 0 (0x0) execution time : 0.011 s
Press any key to continue.
```

Congratulations! You have now written and executed your first C program.

Syntax

You have already seen the following code a couple of times in the first chapters. Let's break it down to understand it better:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!");
  return 0;
}
```

Try it Yourself »

Example explained

Line 1: #include <stdio.h> is a header file library that lets us work with input and output functions, such as printf() (used in line 4). Header files add functionality to C programs.

Don't worry if you don't understand how #include <stdio.h> works. Just think of it as something that (almost) always appears in your program.

Line 2: A blank line. C ignores white space. But we use it to make the code more readable.

Line 3: Another thing that always appear in a C program is main(). This is called a function. Any code inside its curly brackets {} will be executed.

Line 4: printf() is a function used to output/print text to the screen. In our example, it will output "Hello World!".

Note that: Every C statement ends with a semicolon ;

Note: The body of int main() could also been written as:
int main(){printf("Hello World!");return 0;}

Remember: The compiler ignores white spaces. However, multiple lines makes the code more readable.

Line 5: return 0 ends the main() function.

Line 6: Do not forget to add the closing curly bracket } to actually end the main function.

Statements

A computer program is a list of "instructions" to be "executed" by a computer.

In a programming language, these programming instructions are called statements.

The following statement "instructs" the compiler to print the text "Hello World" to the screen:

Example

```
printf("Hello World!");
```

Try it Yourself »

It is important that you end the statement with a semicolon ;

If you forget the semicolon (;), an error will occur and the program will not run:

Example

```
printf("Hello World!")
```

```
error: expected ';' before 'return'
```

Try it Yourself »

Many Statements

Most C programs contain many statements.

The statements are executed, one by one, in the same order as they are written:

Example

```
printf("Hello World!");
printf("Have a good day!");
return 0;
```

Try it Yourself »

Example explained

From the example above, we have three statements:

1. printf("Hello World!");
2. printf("Have a good day!");
3. return 0;

The first statement is executed first (print "Hello World!" to the screen).
Then the second statement is executed (print "Have a good day!" to the screen).
And at last, the third statement is executed (end the C program successfully).

Output (Print Text)

To output values or print text in C, you can use the printf() function:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!");
  return 0;
}
```

Try it Yourself »

Double Quotes

When you are working with text, it must be wrapped inside double quotations marks "".

If you forget the double quotes, an error occurs:

Example

```
printf("This sentence will work!");


printf(This sentence will produce an error.);
```

Try it Yourself »

Many printf Functions

You can use as many printf() functions as you want. However, note that it does not insert a new line at the end of the output:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!");
  printf("I am learning C.");
  printf("And it is awesome!");
  return 0;
}
```

Try it Yourself »

New Lines

To insert a new line, you can use the \n character:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!\n");
  printf("I am learning C.");
  return 0;
}
```

Try it Yourself »

You can also output multiple lines with a single printf() function. However, this could make the code harder to read:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!\nI am learning C.\nAnd it is awesome!");
  return 0;
}
```

Try it Yourself »

Tip: Two \n characters after each other will create a blank line:

Example

```
#include <stdio.h>

int main() {
  printf("Hello World!\n\n");
  printf("I am learning C.");
  return 0;
}
```

Try it Yourself »

What is \n exactly?

The newline character (\n) is called an escape sequence, and it forces the cursor to change its position to the beginning of the next line on the screen. This results in a new line.

Examples of other valid escape sequences are:

Escape Sequence	Description	Try it
\t	Creates a horizontal tab	Try it
\\	Inserts a backslash character (\)	Try it
\"	Inserts a double quote	Try it

character

Comments in C

Comments can be used to explain code, and to make it more readable. It can also be used to prevent execution when testing alternative code.

Comments can be singled-lined or multi-lined.

Single-line Comments

Single-line comments start with two forward slashes (//).

Any text between // and the end of the line is ignored by the compiler (will not be executed).

This example uses a single-line comment before a line of code:

Example

```
// This is a comment
printf("Hello World!");
```

Try it Yourself »

This example uses a single-line comment at the end of a line of code:

Example

```
printf("Hello World!"); // This is a comment
```

Try it Yourself »

C Multi-line Comments

Multi-line comments start with /* and ends with */.

Any text between /* and */ will be ignored by the compiler:

Example

```
/* The code below will print the words Hello World!
to the screen, and it is amazing */
printf("Hello World!");
```

Try it Yourself »

Single or multi-line comments?

It is up to you which you want to use. Normally, we use // for short comments, and /* */ for longer.

Good to know: Before version C99 (released in 1999), you could only use multi-line comments in C.

Variables are containers for storing data values, like numbers and characters.

In C, there are different types of variables (defined with different keywords), for example:

- **int - stores integers (whole numbers), without decimals, such as 123 or -123**
- **float - stores floating point numbers, with decimals, such as 19.99 or -19.99**
- **char - stores single characters, such as 'a' or 'B'. Characters are surrounded by single quotes**

Declaring (Creating) Variables

To create a variable, specify the type and assign it a value:

Syntax

***type variableName* = *value*;**

Where *type* is one of C types (such as int), and *variableName* is the name of the variable (such as x or myName). The equal sign is used to assign a value to the variable.

So, to create a variable that should store a number, look at the following example:

Example

Create a variable called myNum of type int and assign the value 15 to it:

```
int myNum = 15;
```

You can also declare a variable without assigning the value, and assign the value later:

Example

```
// Declare a variable
int myNum;

// Assign a value to the variable
myNum = 15;
```

Output Variables

You learned from the output chapter that you can output values/print text with the printf() function:

Example

```
printf("Hello World!");
```

Try it Yourself »

In many other programming languages (like Python, Java, and C++), you would normally use a print function to display the value of a variable. However, this is not possible in C:

Example

```
int myNum = 15;
printf(myNum);  // Nothing happens
```

Try it Yourself »

To output variables in C, you must get familiar with something called "format specifiers", which you will learn about in the next chapter.

Format Specifiers

Format specifiers are used together with the printf() function to tell the compiler what type of data the variable is storing. It is basically a placeholder for the variable value.

A format specifier starts with a percentage sign %, followed by a character.

For example, to output the value of an int variable, use the format specifier %d surrounded by double quotes (""), inside the printf() function:

Example

```
int myNum = 15;
printf("%d", myNum);  // Outputs 15
```

Try it Yourself »

To print other types, use %c for char and %f for float:

Example

```
// Create variables
int myNum = 15;          // Integer (whole number)
float myFloatNum = 5.99;   // Floating point number
char myLetter = 'D';      // Character
```

```
// Print variables
printf("%d\n", myNum);
printf("%f\n", myFloatNum);
printf("%c\n", myLetter);
```

Try it Yourself »

To combine both text and a variable, separate them with a comma inside the printf() function:

Example

```
int myNum = 15;
printf("My favorite number is: %d", myNum);
```

Try it Yourself »

To print different types in a single printf() function, you can use the following:

Example

```
int myNum = 15;
char myLetter = 'D';
printf("My number is %d and my letter is %c", myNum, myLetter);
```

Try it Yourself »

You will learn more about Data Types in a later chapter.

Print Values Without Variables

You can also just print a value without storing it in a variable, as long as you use the correct format specifier:

Example

```
printf("My favorite number is: %d", 15);
printf("My favorite letter is: %c", 'D');
```

Try it Yourself »

However, it is more sustainable to use variables as they are saved for later and can be re-used whenever.

Change Variable Values

If you assign a new value to an existing variable, it will overwrite the previous value:

Example

```
int myNum = 15;  // myNum is 15
myNum = 10;  // Now myNum is 10
```

Try it Yourself »

You can also assign the value of one variable to another:

Example

```
int myNum = 15;

int myOtherNum = 23;

// Assign the value of myOtherNum (23) to myNum
myNum = myOtherNum;

// myNum is now 23, instead of 15
printf("%d", myNum);
```

Try it Yourself »

Or copy values to empty variables:

Example

```
// Create a variable and assign the value 15 to it
int myNum = 15;

// Declare a variable without assigning it a value
int myOtherNum;

// Assign the value of myNum to myOtherNum
myOtherNum = myNum;

// myOtherNum now has 15 as a value
printf("%d", myOtherNum);
```

Try it Yourself »

Add Variables Together

To add a variable to another variable, you can use the + operator:

Example

```
int x = 5;
int y = 6;
int sum = x + y;
printf("%d", sum);
```

Try it Yourself »

Declare Multiple Variables

To declare more than one variable of the same type, use a comma-separated list:

Example

```
int x = 5, y = 6, z = 50;
printf("%d", x + y + z);
```

Try it Yourself »

You can also assign the same value to multiple variables of the same type:

Example

```
int x, y, z;
x = y = z = 50;
printf("%d", x + y + z);
```

Real-Life Example

Often in our examples, we simplify variable names to match their data type (myInt or myNum for int types, myChar for char types, and so on). This is done to avoid confusion.

However, for a practical example of using variables, we have created a program that stores different data about a college student:

Example

```
// Student data
int studentID = 15;
int studentAge = 23;
float studentFee = 75.25;
char studentGrade = 'B';

// Print variables
printf("Student id: %d\n", studentID);
printf("Student age: %d\n", studentAge);
printf("Student fee: %f\n", studentFee);
printf("Student grade: %c", studentGrade);
```

Try it Yourself »

Calculate the Area of a Rectangle

In this real-life example, we create a program to calculate the area of a rectangle (by multiplying the length and width):

Example

```
// Create integer variables
int length = 4;
int width = 6;
int area;
```

```
// Calculate the area of a rectangle
area = length * width;

// Print the variables
printf("Length is: %d\n", length);
printf("Width is: %d\n", width);
printf("Area of the rectangle is: %d", area);
```

Try it Yourself »

Data Types

As explained in the Variables chapter, a variable in C must be a specified data type, and you must use a format specifier inside the printf() function to display it:

Example

```
// Create variables
int myNum = 5;           // Integer (whole number)
float myFloatNum = 5.99;   // Floating point number
char myLetter = 'D';      // Character

// Print variables
printf("%d\n", myNum);
printf("%f\n", myFloatNum);
printf("%c\n", myLetter);
```

Try it Yourself »

Basic Data Types

The data type specifies the size and type of information the variable will store.

In this tutorial, we will focus on the most basic ones:

Data Type	Size	Description	Example
int	2 or 4 bytes	Stores whole numbers, without decimals	1
float	4 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 6-7 decimal digits	1.99
double	8 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits	1.99
char	1 byte	Stores a single character/letter/number, or ASCII values	'A'

Basic Format Specifiers

There are different format specifiers for each data type. Here are some of them:

Format Specifier	Data Type	Try it
%d or %i	int	Try it »
%f or %F	float	Try it »
%lf	double	Try it »
%c	char	Try it »
%s	Used for strings (text), which you will learn more about in a later chapter	Try it »

Note: It is important that you use the correct format specifier for the specified data type, or the program may produce errors or even crash.

The char Type

The char data type is used to store a single character.

The character must be surrounded by single quotes, like 'A' or 'c', and we use the %c format specifier to print it:

Example

```
char myGrade = 'A';
printf("%c", myGrade);
```

Try it Yourself »

Alternatively, if you are familiar with ASCII, you can use ASCII values to display certain characters. Note that these values are not surrounded by quotes (''), as they are numbers:

Example

```
char a = 65, b = 66, c = 67;
printf("%c", a);
printf("%c", b);
printf("%c", c);
```

Try it Yourself »

Tip: A list of all ASCII values can be found in our ASCII Table Reference.

Notes on Characters

If you try to store more than a single character, it will only print the last character:

Example

```
char myText = 'Hello';
printf("%c", myText);
```

Try it Yourself »

Note: Don't use the char type for storing multiple characters, as it may produce errors.

To store multiple characters (or whole words), use strings (which you will learn more about in a later chapter):

Example

```
char myText[] = "Hello";
printf("%s", myText);
```

Try it Yourself »

For now, just know that we use strings for storing multiple characters/text, and the char type for single characters

Numeric Types

Use int when you need to store a whole number without decimals, like 35 or 1000, and float or double when you need a floating point number (with decimals), like 9.99 or 3.14515.

int

```
int myNum = 1000;
printf("%d", myNum);
```

Try it Yourself »

float

```
float myNum = 5.75;
printf("%f", myNum);
```

Try it Yourself »

double

```
double myNum = 19.99;
printf("%lf", myNum);
```

Try it Yourself »

float vs. double

The precision of a floating point value indicates how many digits the value can have after the decimal point. The precision of float is six or seven decimal digits, while double variables have a precision of about 15 digits. Therefore, it is often safer to use double for most calculations - but note that it takes up twice as much memory as float (8 bytes vs. 4 bytes).

Scientific Numbers

A floating point number can also be a scientific number with an "e" to indicate the power of 10:

Example

```
float f1 = 35e3;
double d1 = 12E4;

printf("%f\n", f1);
printf("%lf", d1);
```

Try it Yourself »

Set Decimal Precision

You have probably already noticed that if you print a floating point number, the output will show many digits after the decimal point:

Example

```
float myFloatNum = 3.5;
double myDoubleNum = 19.99;
```

Get the Memory Size

We introduced in the data types chapter that the memory size of a variable

Data Type	Size
int	2 or 4 bytes
float	4 bytes
double	8 bytes
char	1 byte

The memory size refers to how much space a type occupies in the *computer's memory*.

To actually get the size (in bytes) of a data type or variable, use the sizeof operator:

Example

```
int myInt;
float myFloat;
double myDouble;
char myChar;

printf("%lu\n", sizeof(myInt));
printf("%lu\n", sizeof(myFloat));
printf("%lu\n", sizeof(myDouble));
printf("%lu\n", sizeof(myChar));
```

Try it Yourself »

Note that we use the %lu format specifer to print the result, instead of %d. It is because the compiler expects the sizeof operator to return a long unsigned int (%lu), instead of int (%d). On some computers it might work with %d, but it is safer to use %lu.

Why Should I Know the Size of Data Types?

Knowing the size of different data types is important because it says something about memory usage and performance.

For example, the size of a char type is 1 byte. Which means if you have an array of 1000 char values, it will occupy 1000 bytes (1 KB) of memory.

Using the right data type for the right purpose will save memory and improve the performance of your program.

You will learn more about the sizeof operator later in this tutorial, and how to use it in different scenarios.

```
printf("%f\n", myFloatNum); // Outputs 3.500000
printf("%lf", myDoubleNum); // Outputs 19.990000
```

Try it Yourself »

If you want to remove the extra zeros (set decimal precision), you can use a dot (.) followed by a number that specifies how many digits that should be shown after the decimal point:

Example

```
float myFloatNum = 3.5;

printf("%f\n", myFloatNum);   // Default will show 6 digits after the decimal point
printf("%.1f\n", myFloatNum); // Only show 1 digit
printf("%.2f\n", myFloatNum); // Only show 2 digits
printf("%.4f", myFloatNum);   // Only show 4 digits
```

Try it Yourself »

Real-Life Example

Here's a real-life example of using different data types, to calculate and output the total cost of a number of items:

Example

```
// Create variables of different data types
int items = 50;
float cost_per_item = 9.99;
float total_cost = items * cost_per_item;
char currency = '$';

// Print variables
printf("Number of items: %d\n", items);
printf("Cost per item: %.2f %c\n", cost_per_item, currency);
printf("Total cost = %.2f %c\n", total_cost, currency);
```

Try it Yourself »

Type Conversion

Sometimes, you have to convert the value of one data type to another type. This is known as type conversion.

For example, if you try to divide two integers, 5 by 2, you would expect the result to be 2.5. But since we are working with integers (and not floating-point values), the following example will just output 2:

Example

```
int x = 5;
int y = 2;
int sum = 5 / 2;

printf("%d", sum); // Outputs 2
```

Try it Yourself »

To get the right result, you need to know how type conversion works.

There are two types of conversion in C:

- Implicit Conversion (automatically)
- Explicit Conversion (manually)

Implicit Conversion

Implicit conversion is done automatically by the compiler when you assign a value of one type to another.

For example, if you assign an int value to a float type:

Example

```
// Automatic conversion: int to float
float myFloat = 9;

printf("%f", myFloat); // 9.000000
```

Try it Yourself »

As you can see, the compiler automatically converts the int value 9 to a float value of 9.000000.

This can be risky, as you might lose control over specific values in certain situations.

Especially if it was the other way around - the following example automatically converts the float value 9.99 to an int value of 9:

Example

```
// Automatic conversion: float to int
int myInt = 9.99;

printf("%d", myInt); // 9
```

Try it Yourself »

What happened to .99? We might want that data in our program! So be careful. It is important that you know how the compiler work in these situations, to avoid unexpected results.

As another example, if you divide two integers: 5 by 2, you know that the sum is 2.5. And as you know from the beginning of this page, if you store the sum as an integer, the result will only display the number 2. Therefore, it would be better to store the sum as a float or a double, right?

Example

```
float sum = 5 / 2;

printf("%f", sum); // 2.000000
```

Try it Yourself »

Why is the result 2.00000 and not 2.5? Well, it is because 5 and 2 are still integers in the division. In this case, you need to manually convert the integer values to floating-point values. (see below).

Explicit Conversion

Explicit conversion is done manually by placing the type in parentheses () in front of the value.

Considering our problem from the example above, we can now get the right result:

Example

```
// Manual conversion: int to float
float sum = (float) 5 / 2;

printf("%f", sum); // 2.500000
```

Try it Yourself »

You can also place the type in front of a variable:

Example

```
int num1 = 5;
int num2 = 2;
float sum = (float) num1 / num2;

prints("%f", sum); // 2.500000
```

Try it Yourself »

And since you learned about "decimal precision" in the previous chapter, you could make the output even cleaner by removing the extra zeros (if you like):

Example

```
int num1 = 5;
int num2 = 2;
float sum = (float) num1 / num2;

printf("%.1f", sum); // 2.5
```

Try it Yourself »

Real-Life Example

Here's a real-life example of data types and type conversion where we create a program to calculate the percentage of a user's score in relation to the maximum score in a game:

Example

```
// Set the maximum possible score in the game to 500
int maxScore = 500;

// The actual score of the user
int userScore = 423;

/* Calculate the percantage of the user's score in relation to the maximum
available score.
Convert userScore to float to make sure that the division is accurate */
float percentage = (float) userScore / maxScore * 100.0;

// Print the percentage
printf("User's percentage is %.2f", percentage);
```